Session 4

Informed Search
Part I

Arash Marioriyad
18 Farvardin 1405

CE 417
Artificial Intelligence
Sharif Univ. of Tech.

Some of Slides have been adopted from Berkeley CS188

Up To Now …

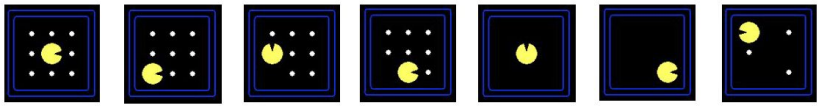
- Search Problem
- Uninformed Search Algorithms
  - BFS
  - DFS
  - UCS
- Heuristic

Today: Informed Search

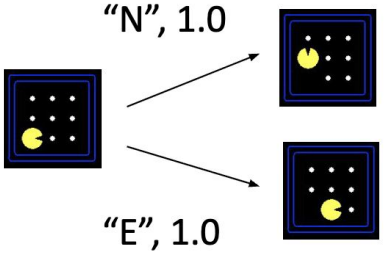
- Heuristic
- Greedy Search
- A* Search
- Admissible Heuristic

Search Problems

- A search problem consists of:
  - A state space



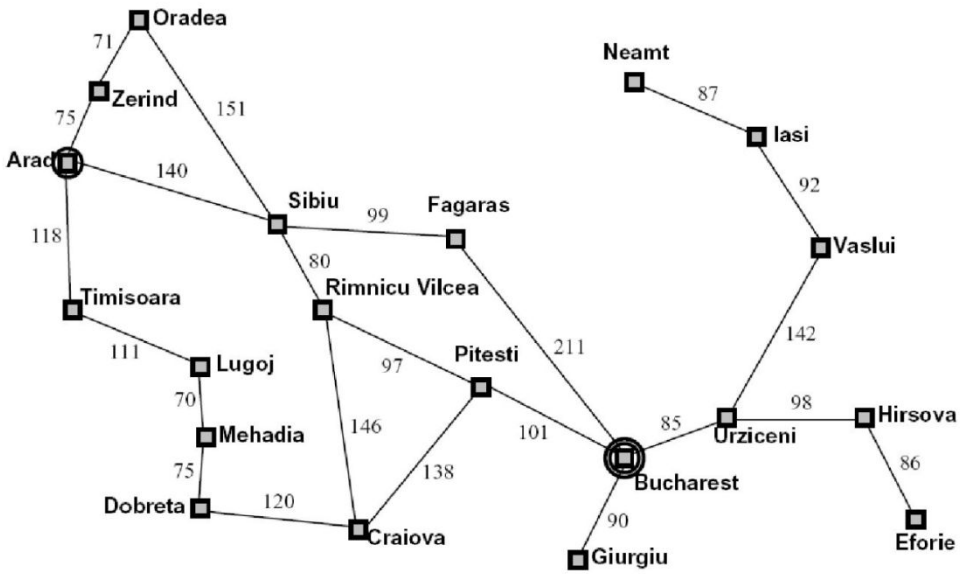
  - A successor function (with actions, costs)



  - A start state and a goal test

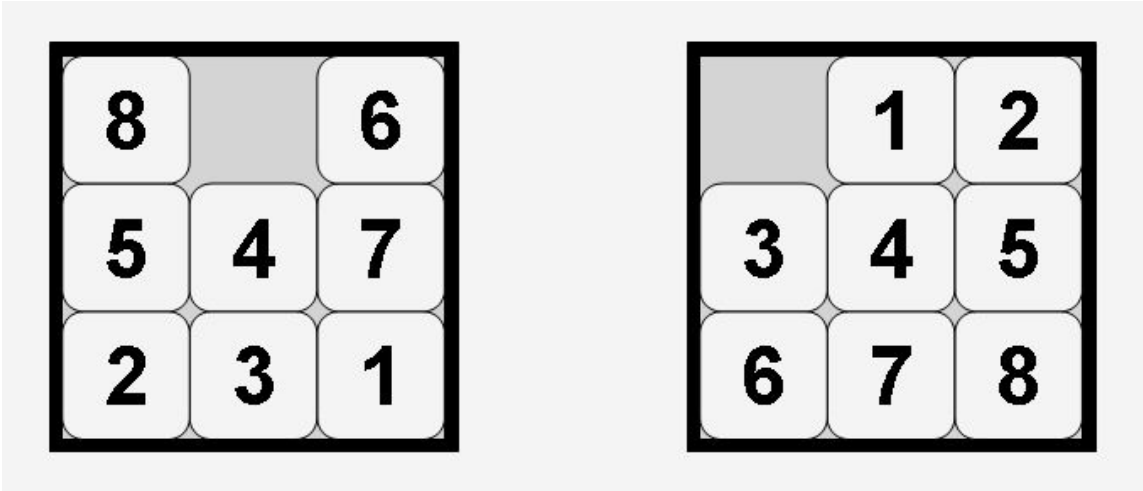
- A solution is a sequence of actions (a plan) which transforms the start state to a goal state

Example: Traveling in Romania

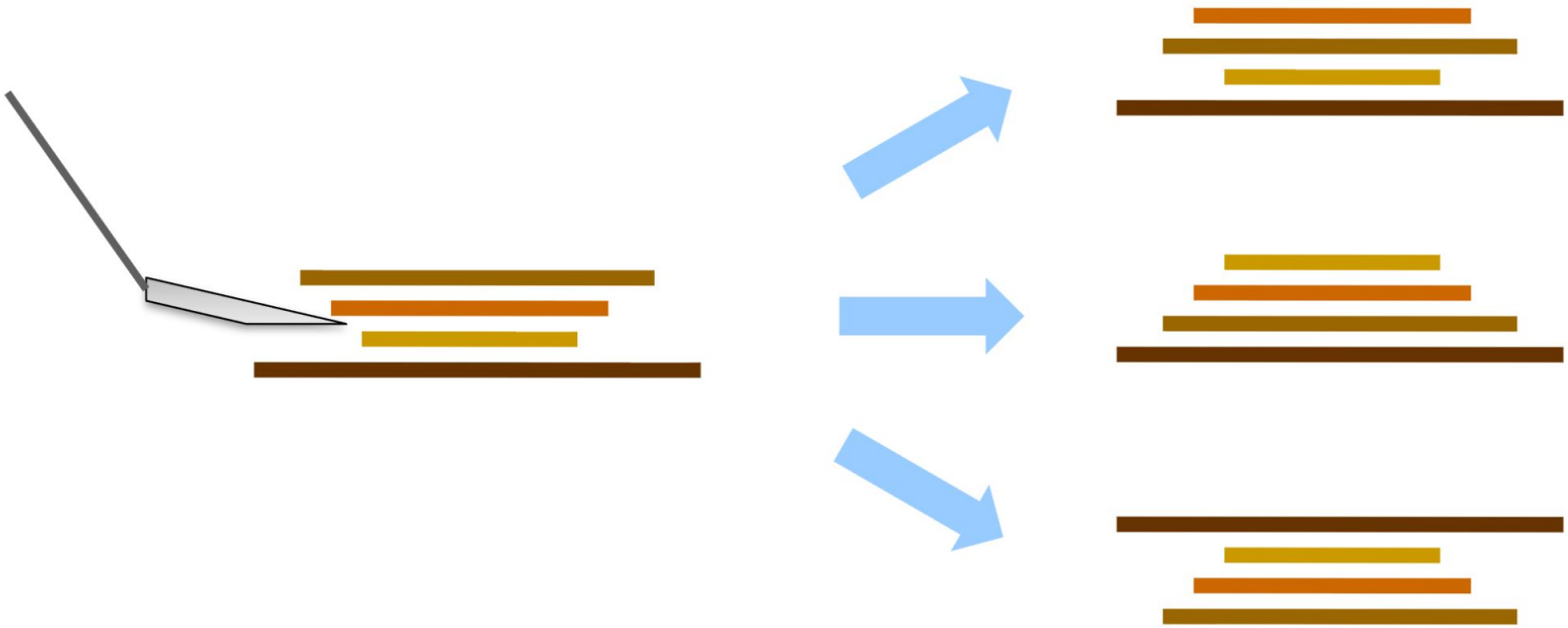


- State space:
  - Cities
- Successor function:
  - Roads: Go to adjacent city with cost = distance
- Start state:
  - Arad
- Goal test:
  - Is state == Bucharest?
- Solution?

Eight Puzzle

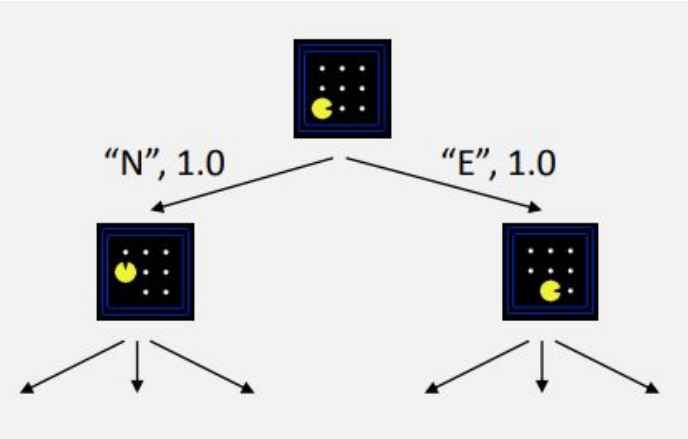


Pancake Problem



Tree Search

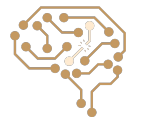
- A general framework for search algorithms
- Nodes are plans (sequence of states obtained by applying actions)
- Plans have costs
- We may see a state multiple times



Search Algorithms

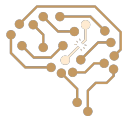
- What do search algorithms tell us?
  - How to build the search tree?
  - How to select the next element (unexplored partial plans) from fringe?

- What don’t search algorithms tell us?
  - In what order add the partial plans to fringe?
  - When to apply goal test? expansion time or at **dequeuing time**?



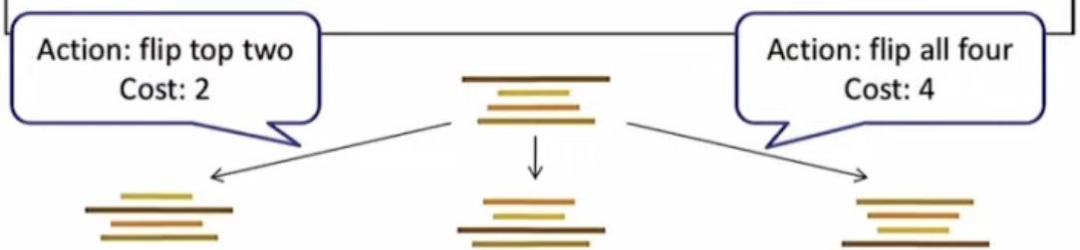
Tree Search

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```



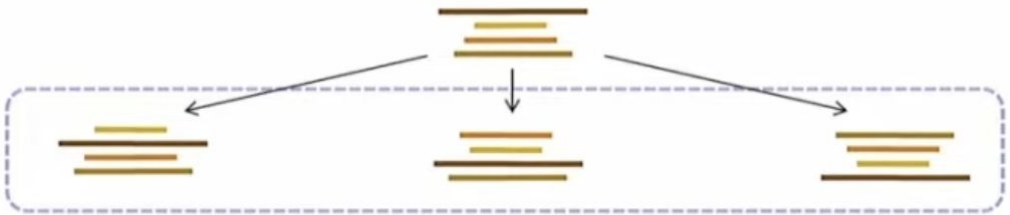
Tree Search

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```



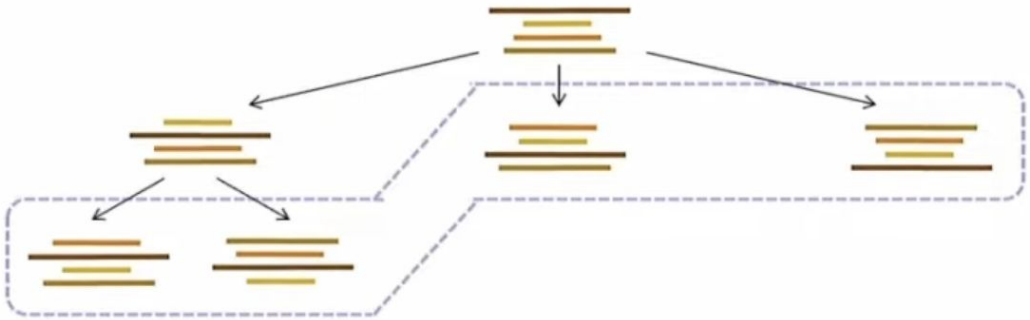
Tree Search

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```



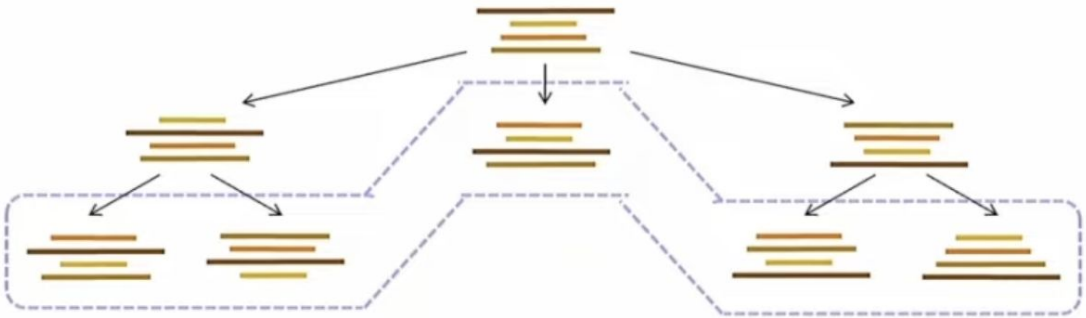
Tree Search

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```



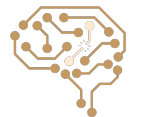
Tree Search

```
function TREE-SEARCH( problem, strategy) returns a solution, or failure
    initialize the search tree using the initial state of problem
    loop do
        if there are no candidates for expansion then return failure
        choose a leaf node for expansion according to strategy
        if the node contains a goal state then return the corresponding solution
        else expand the node and add the resulting nodes to the search tree
    end
```



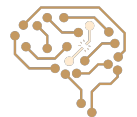
Search Algorithms

- **Complete:** If it is guaranteed to find a solution whenever one exists.
- **Optimal:** If it is guaranteed to find the best (lowest-cost) solution among all possible ones.
- **Time Complexiy**
- **Space Complexity**



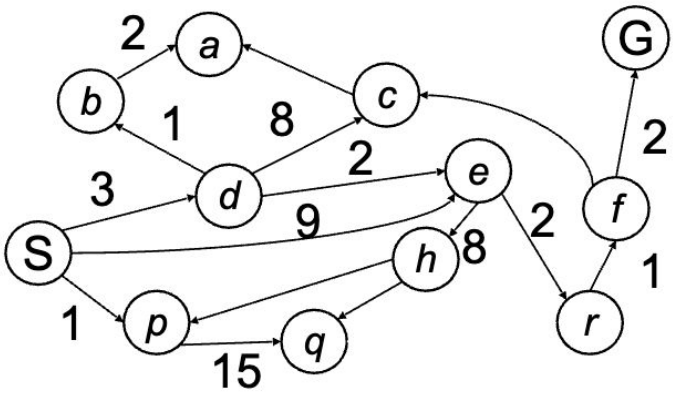
Uninformed Search Algorithms

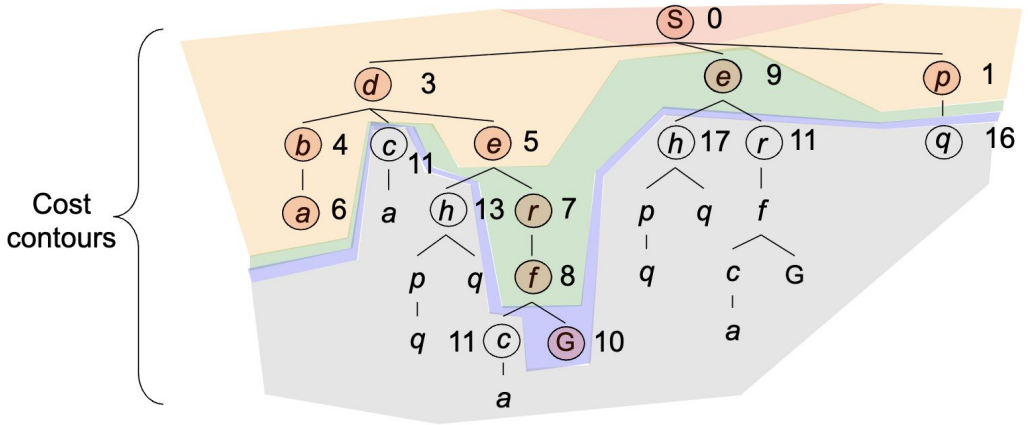
- **Breadth-First Search (BFS):**
  - Dequeues nodes in order of shallowest depth first (FIFO queue).
- **Depth-First Search (DFS):**
  - Dequeues the most recently added node first (LIFO stack).
- **Iterative Deepening Search (IDS):**
  - Repeatedly runs DFS, increasing the depth limit each time.
- **Uniform Cost Search (UCS):**
  - Dequeues the node with the lowest cumulative path cost (g(n)) first.



UCS Algorithm

- g(n) = cost from root to n
- Strategy: expand lowest g(n)
- Frontier is a priority queue sorted by g(n)





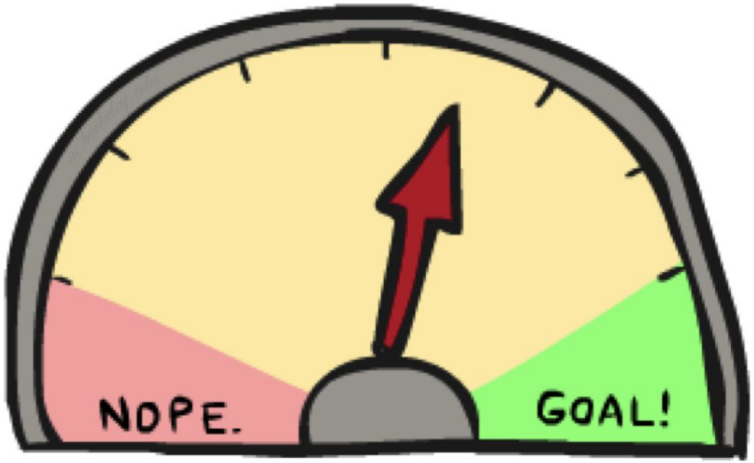
UCS Explores Everywhere!

- UCS explores all plans with cost <k before explore a plan of cost k

Uniform-Cost Search

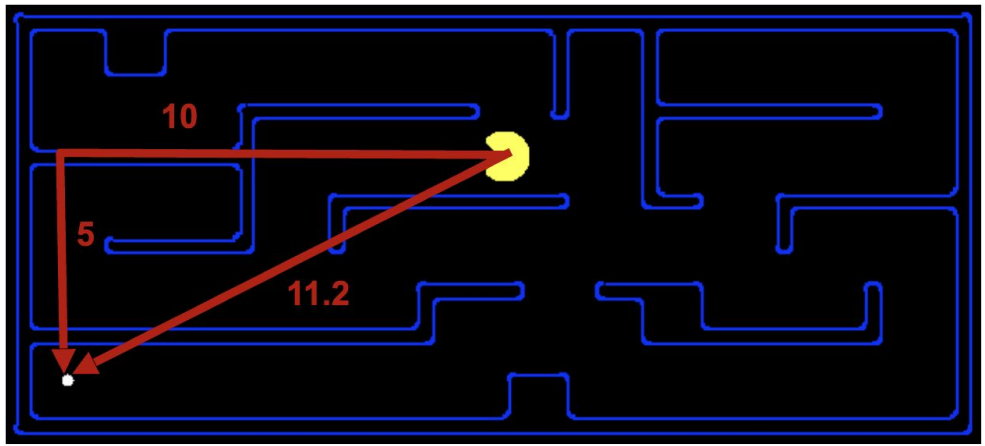
Informed Search

- **Greedy**
- **A***



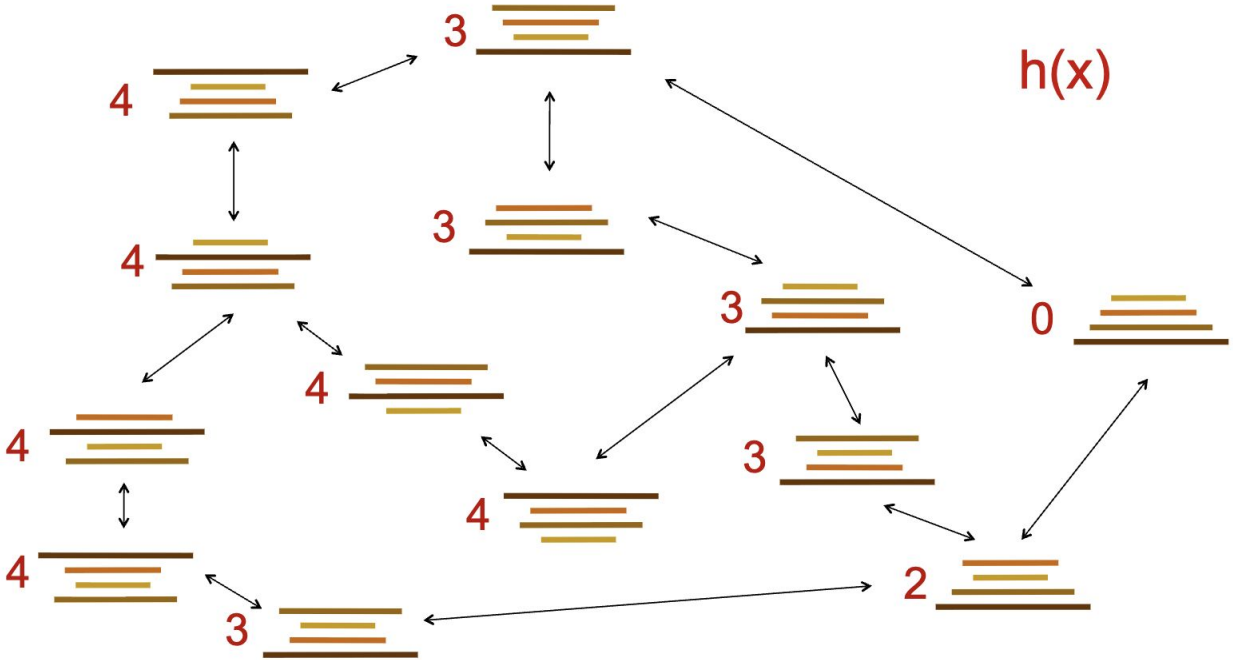
Search Heuristics

- A function h(s): **estimates** how close a state is to a goal
- For each search problem we design/consider particular heuristics
- Example:
  - Manhattan distance
  - Euclidean distance



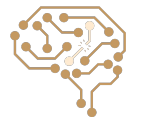
Search Heuristic - Pancake Example

- Heuristic: the number of the largest pancake that is still out of place



Search Heuristic

- Particular heuristics for different search problems
- What if the goal state changes?
- What if the start state changes?
- What if the cost between states changes?

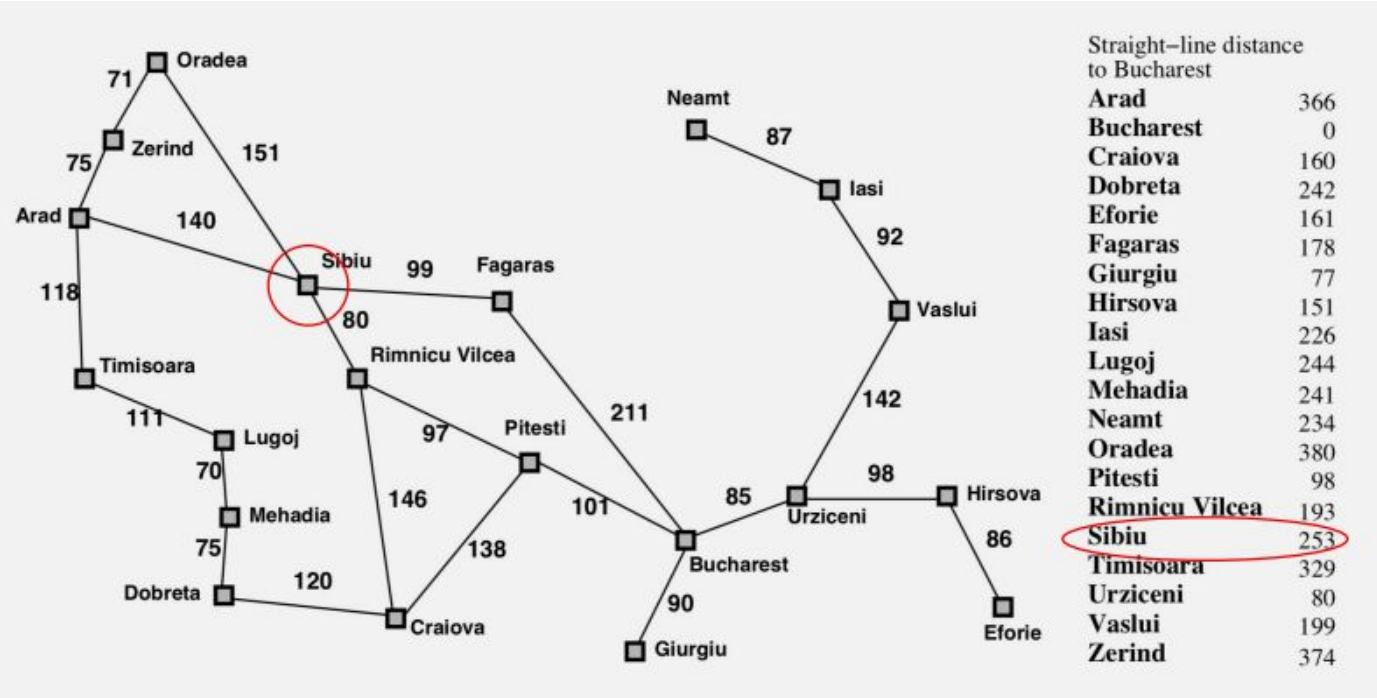


Greedy Search

- Expand the node that have the best heuristic (Best First Search)

State Space Graph - Romania Routing Problem

- Heuristic: Expand the node that seems closest (the lowest h(s))



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

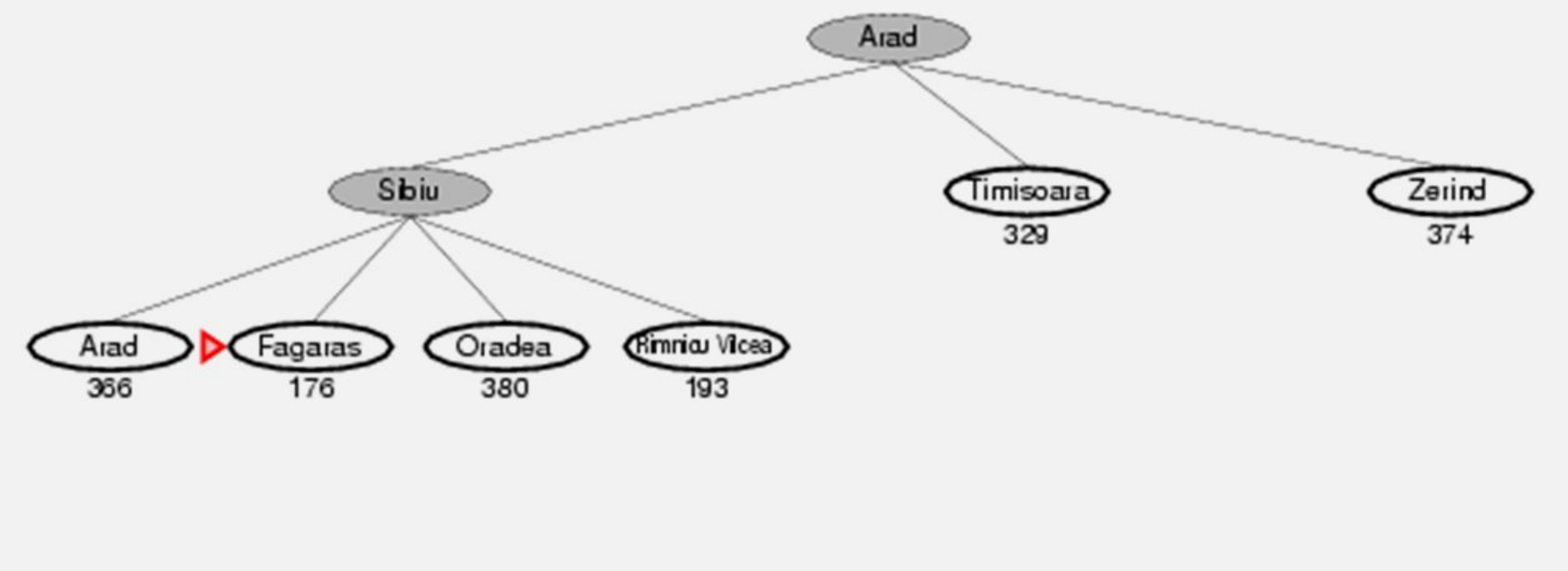
Greedy Best-first Search Example

Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

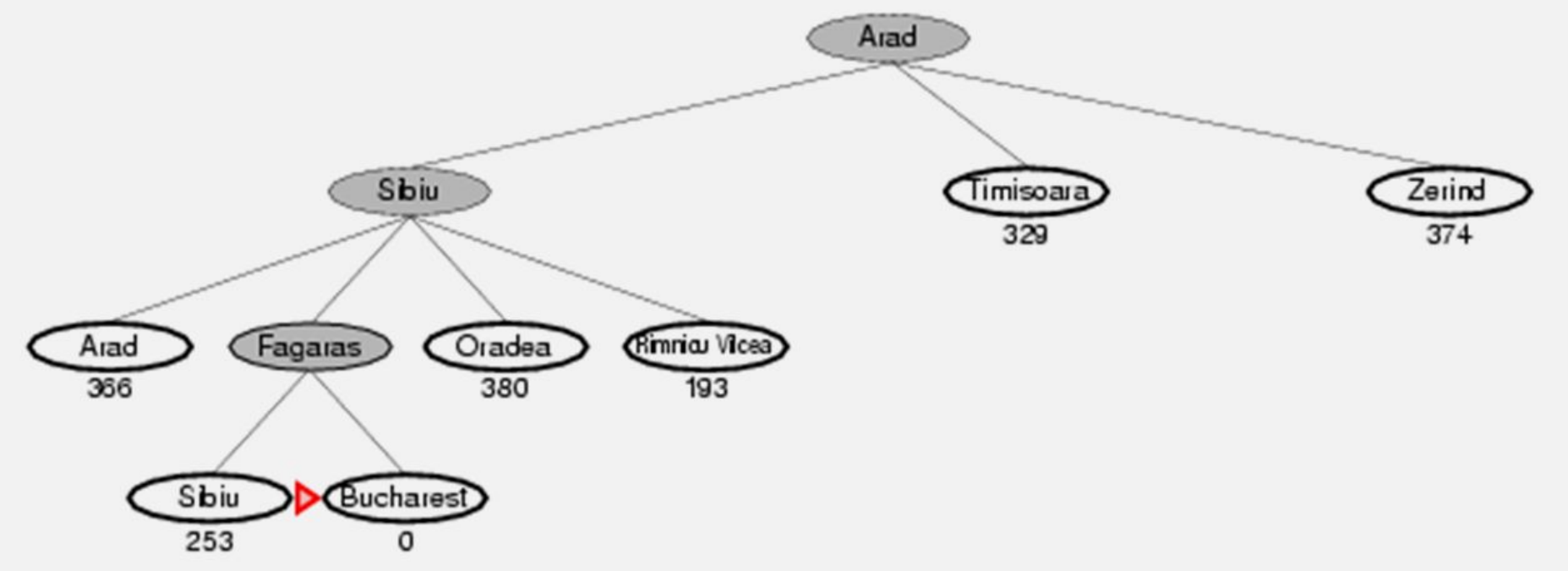
Greedy Best-first Search Example

Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Best-first Search Example

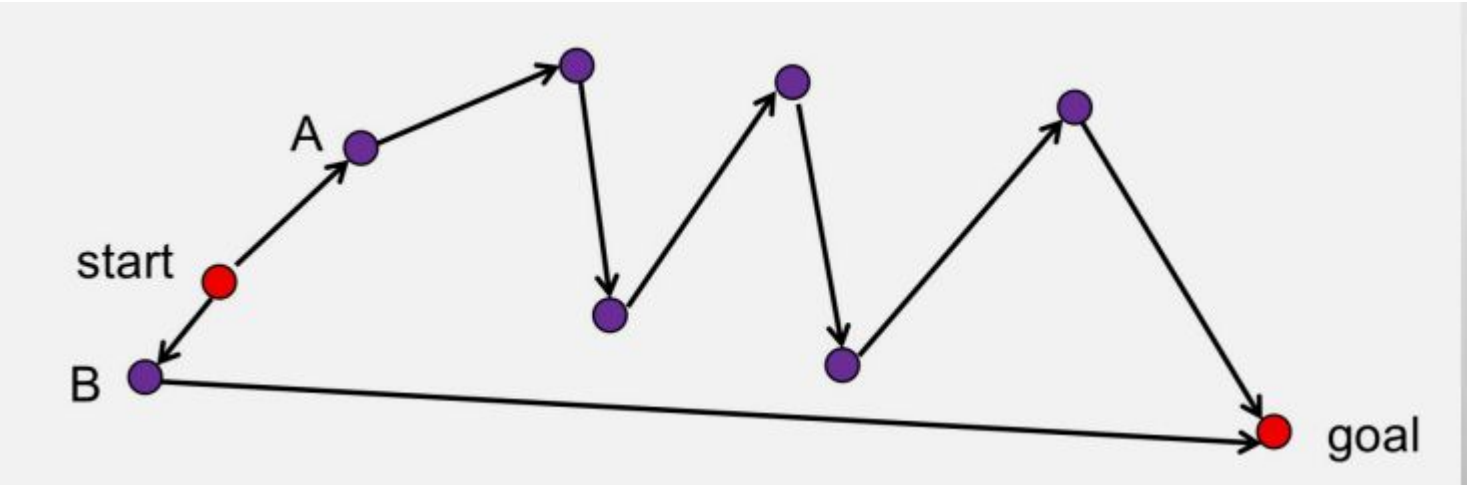


Greedy Best-first Search Example

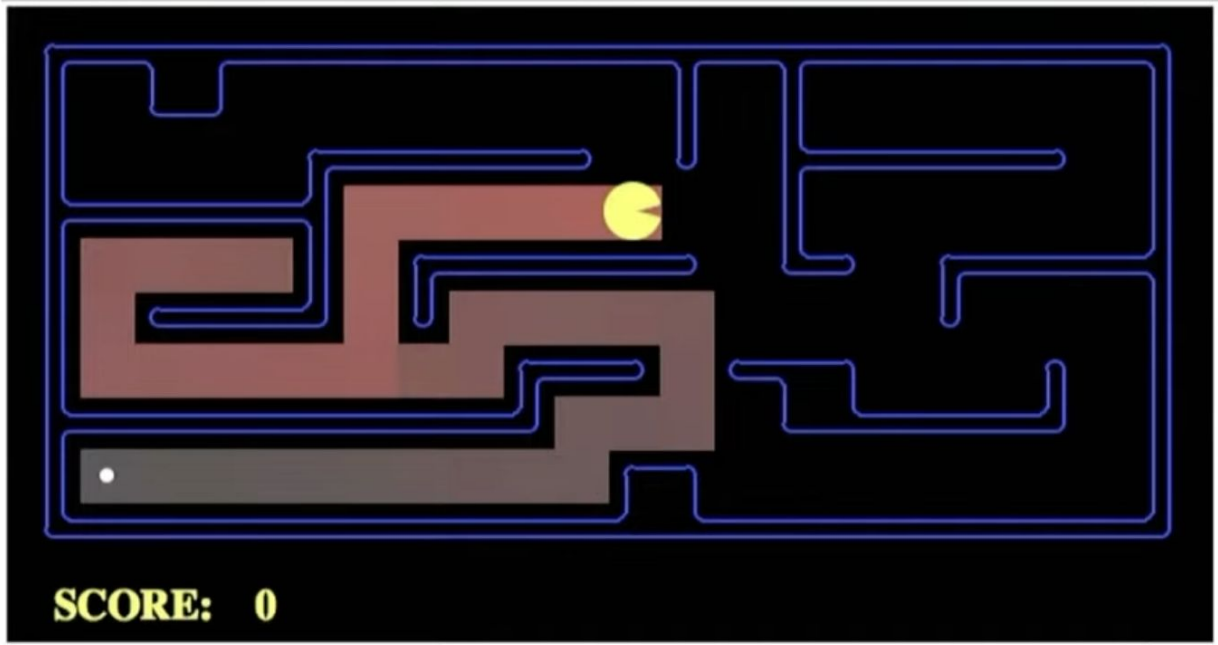


How Can It Go Wrong?

- Heuristic: Expand the node that seems closest...

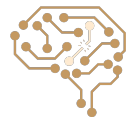


How Can It Go Wrong? - Pacman Small Maze

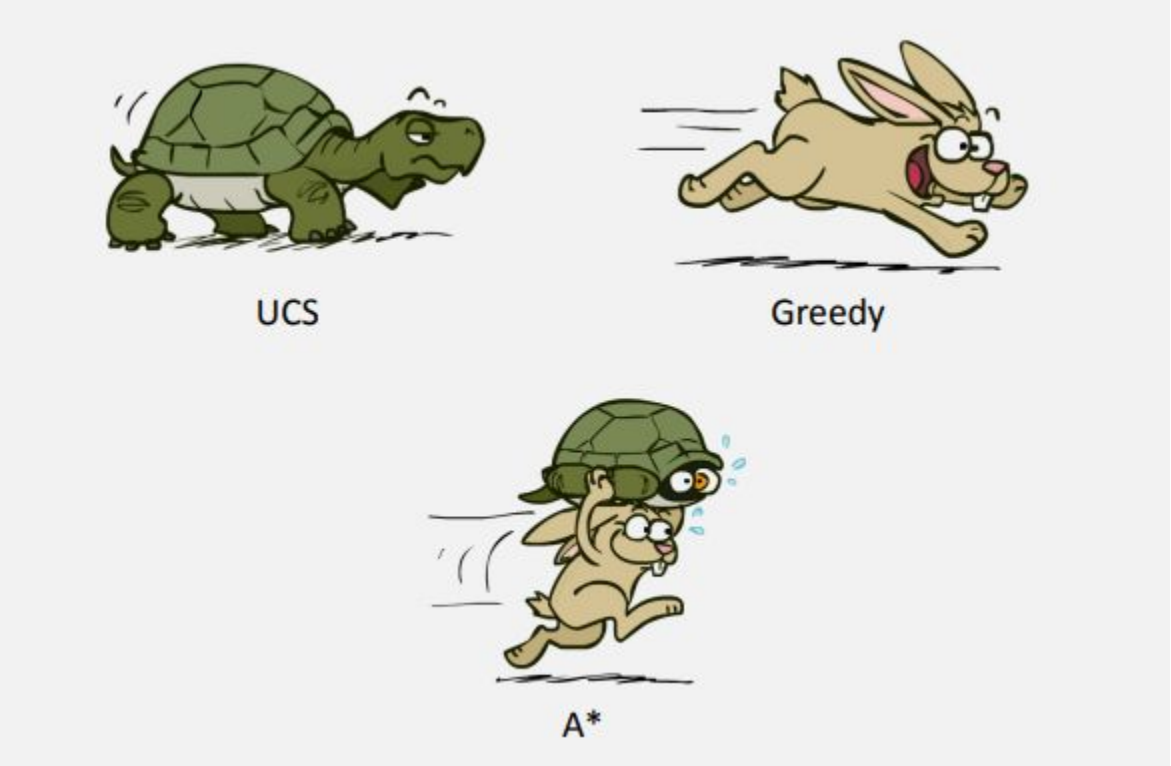


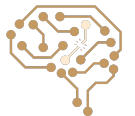
Greedy Search Characteristics

- Greedy search is sub-optimal
- Worst Case: Like DFS, it may explore everything
- Completeness (like DFS):
  - With cycle checking
  - Finite number of states

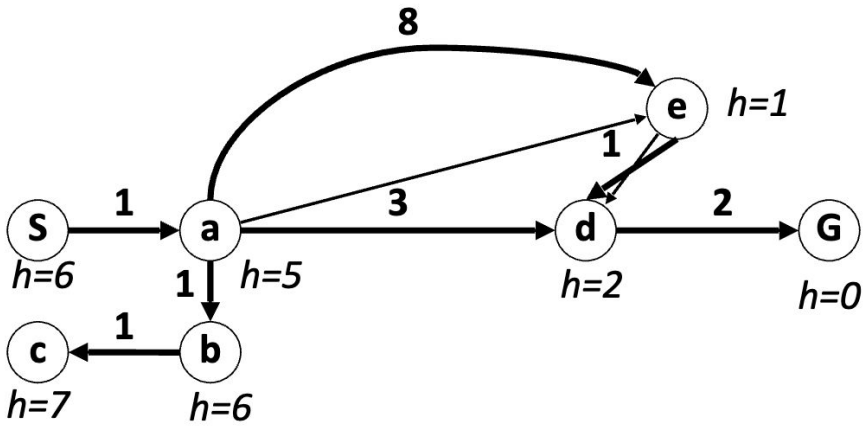


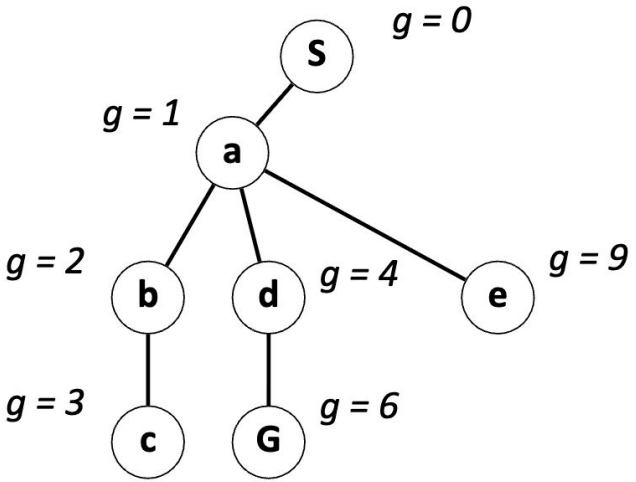
Search Algorithms So Far



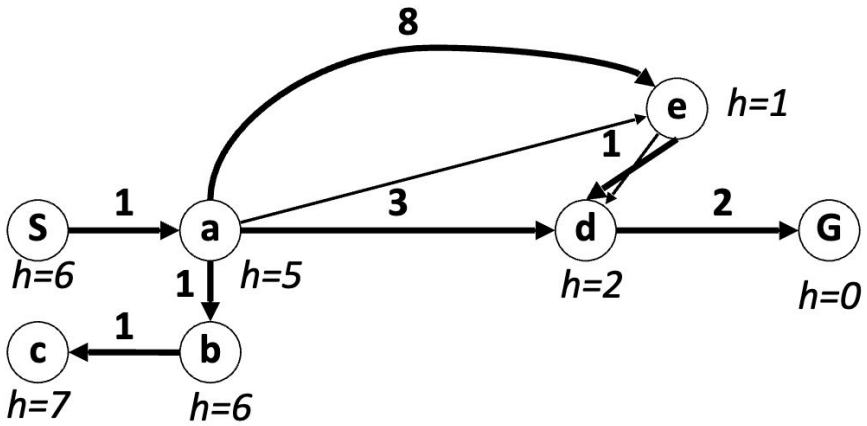


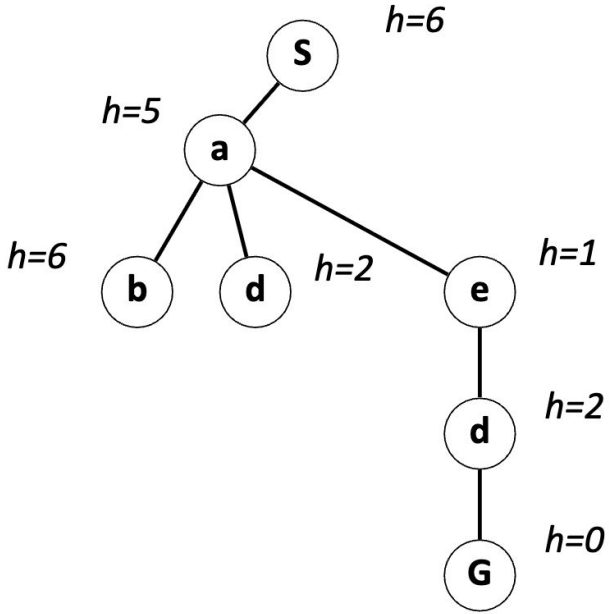
UCS Example





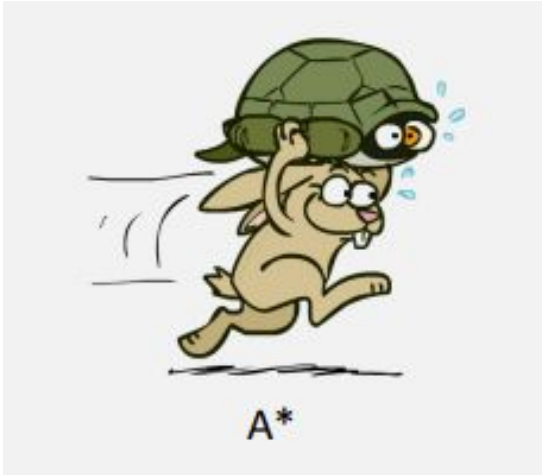
Greedy Example



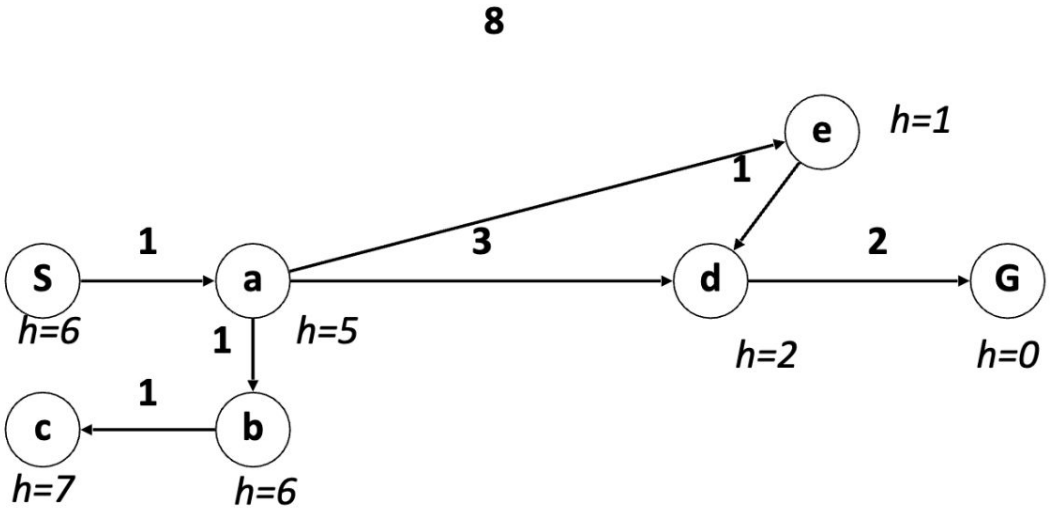


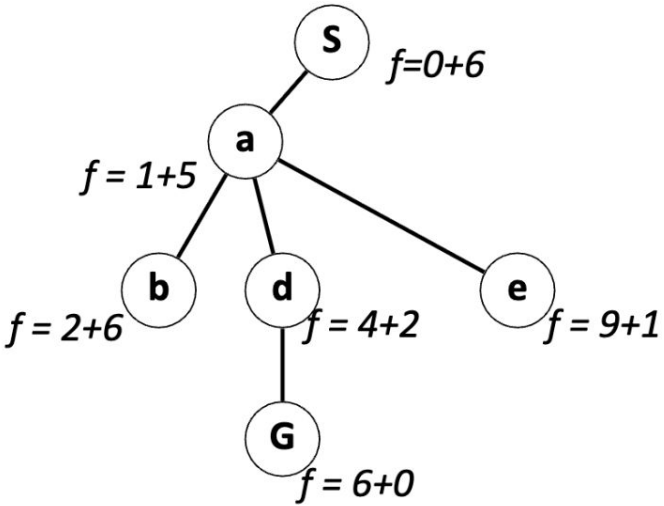
A* Search - Combining UCS and Greedy

- Best first search with f(n) = g(n) + h(n)
- g(n) = sum of costs from start to n
- h(n) = estimate of the cost of path n → goal
- h(goal) = 0
- g(n) ~ uniform cost search
- h(n) ~ greedy search
- Best of both worlds ...

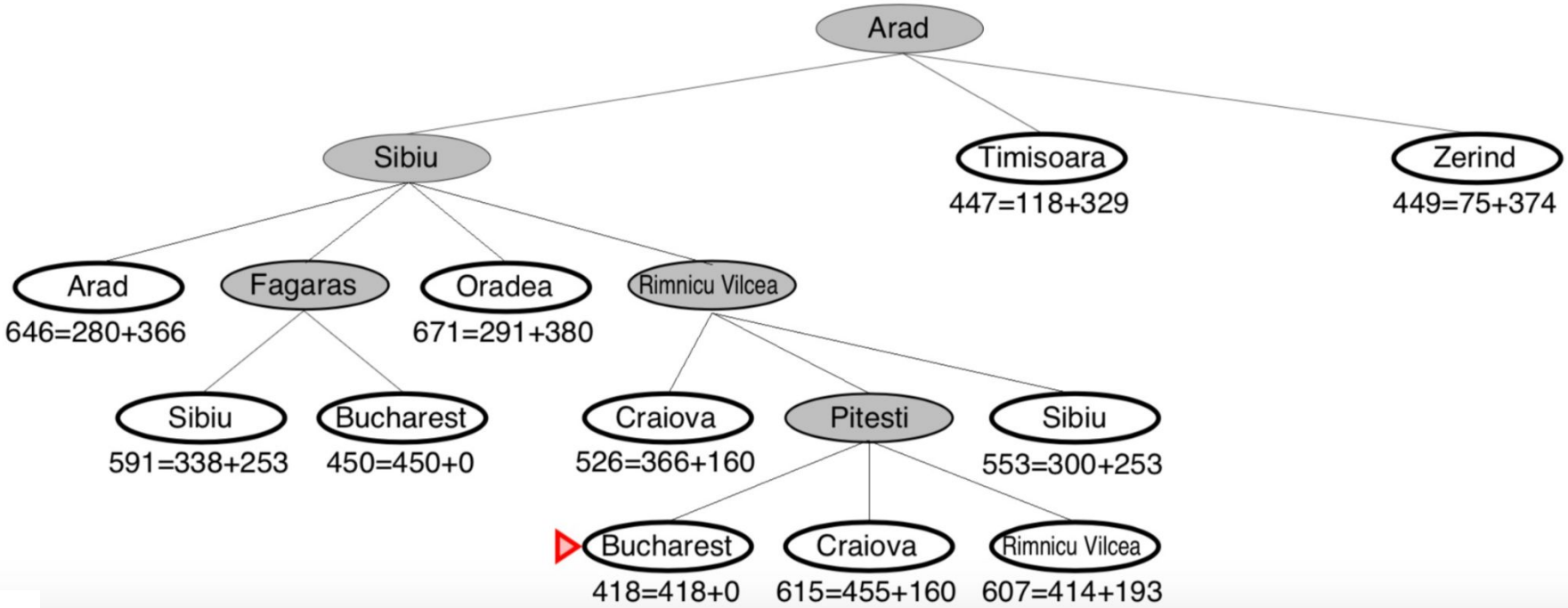


A* Search - Example

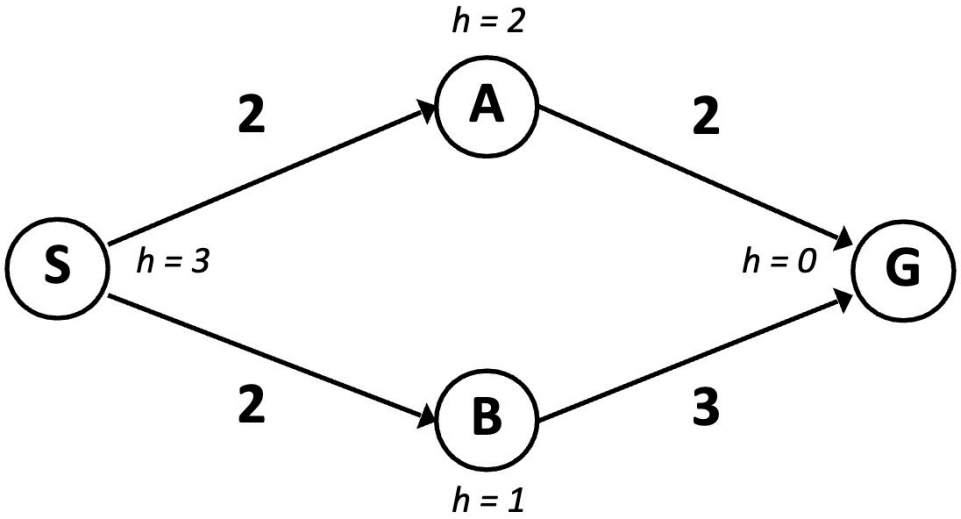


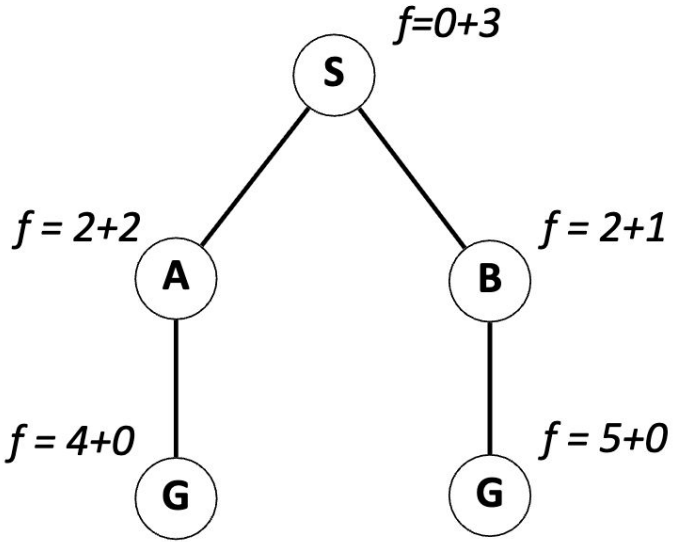


A* Search - Romania Routing Example



When Should A* Search Terminate?

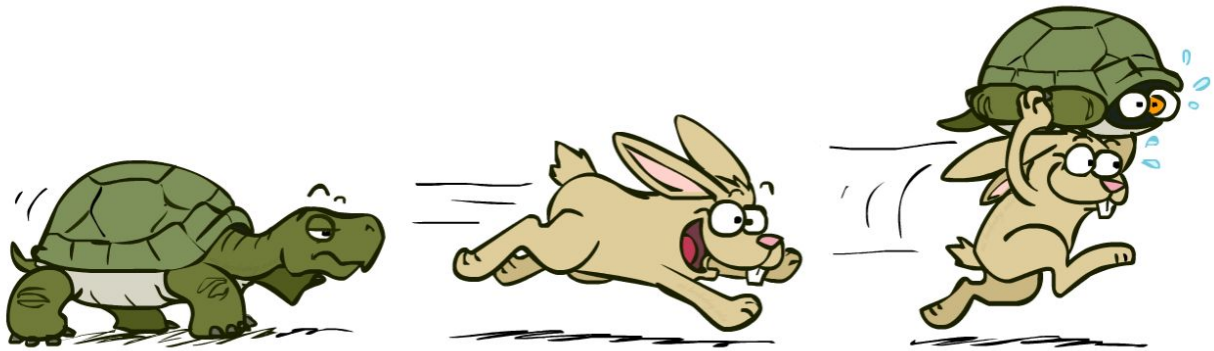




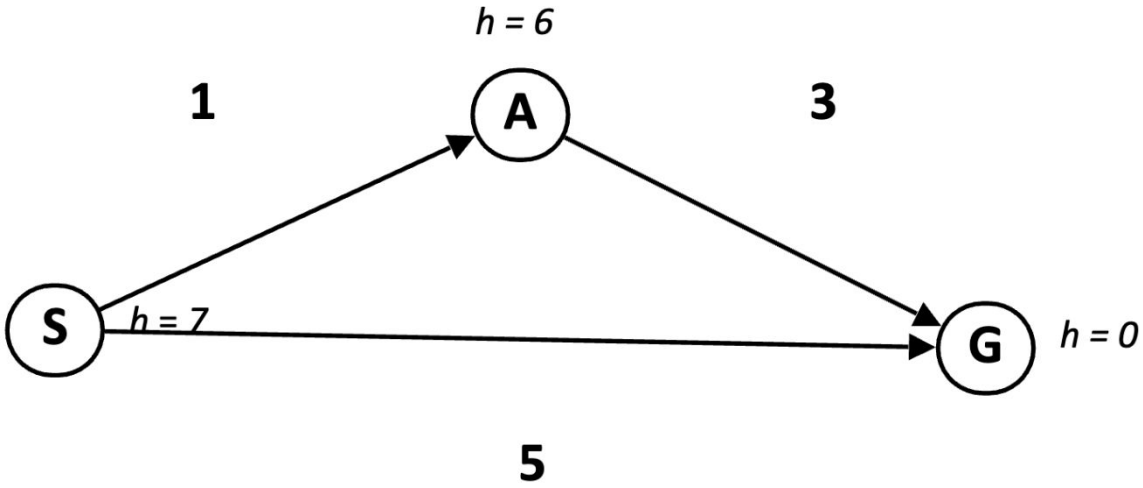
- Only stop when we dequeue a goal

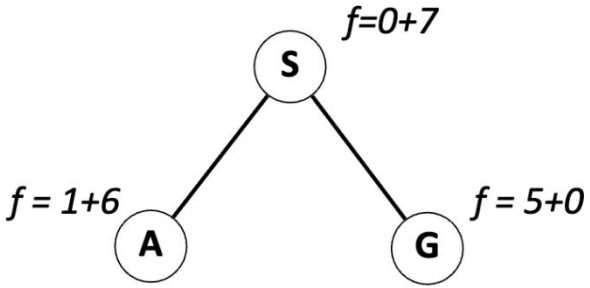
Reflection

- Where does the heuristic h comes from?
- How one proves that A* is optimal?
- Does A* expand minimum possible nodes before termination?



Is A* Search Optimal?

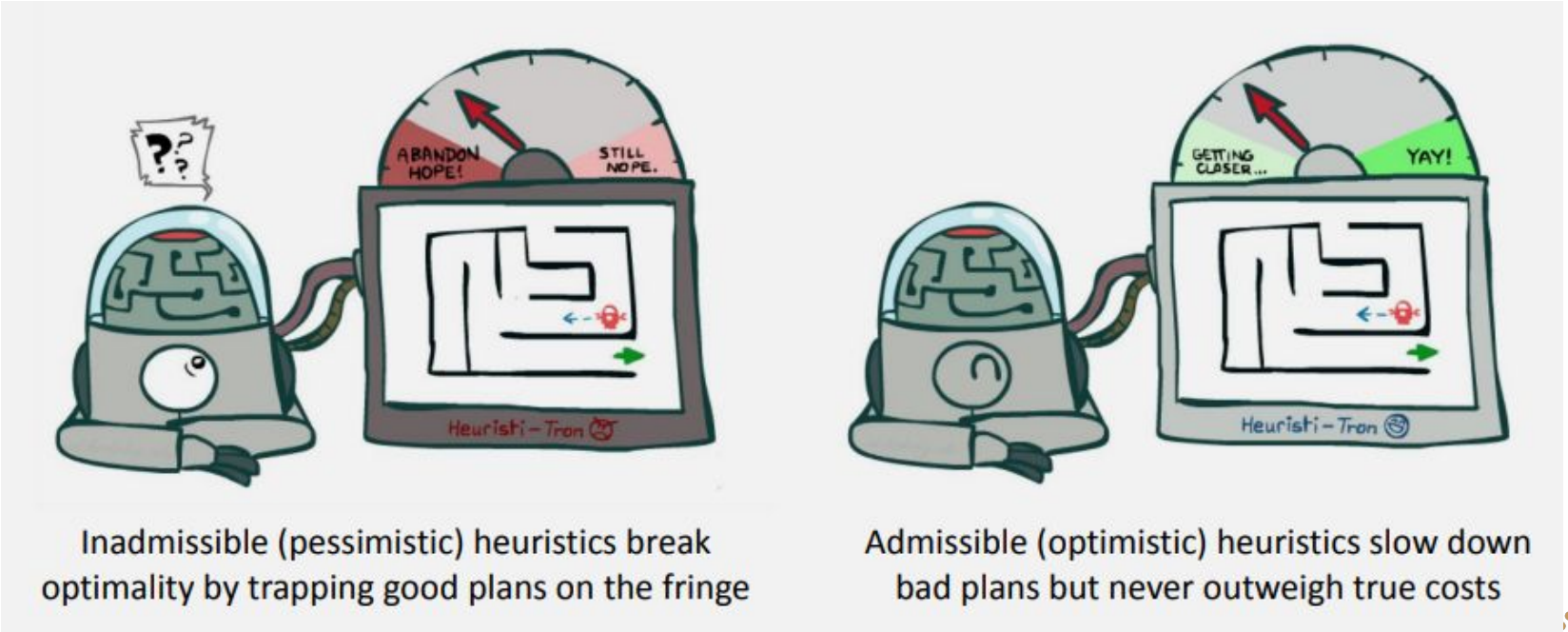




- What went wrong?
- Estimated good goal cost > Actual goal cost

A* optimality (tree-search)?

- Theorem: If h(n) is admissible then A* is optimal in tree search



Inadmissible (pessimistic) heuristics break optimality by trapping good plans on the fringe

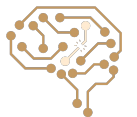
Admissible (optimistic) heuristics slow down bad plans but never outweigh true costs

Admissible Heuristics

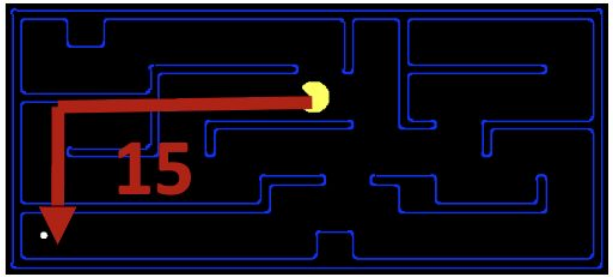
- A heuristic h is admissible (optimistic) if for every node n:

$$0 \leq h(n) \leq h^*(n)$$

- Where $h^*(n)$ is the true cost to a nearest goal
- Always underestimate the true cost to goal



Admissible Heuristics - Examples

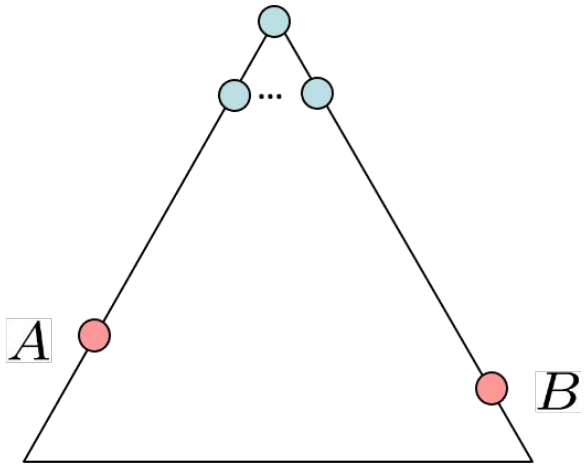


4

A* Optimality in Tree Search?

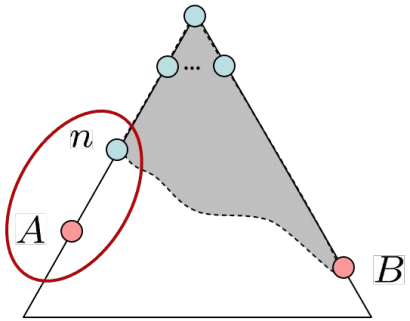
- Assume:
  - A is an optimal goal node
  - B is a suboptimal goal node
  - h is admissible

- Claim:
  - A will exit the fringe before B



A* Optimality in Tree Search?

- Imagine B is on the fringe
- Some ancestor n of A is on the fringe,
- Maybe A itself is on the fringe too
- Claim: n will be expanded before B
  - f(n) is less or equal to f(A)



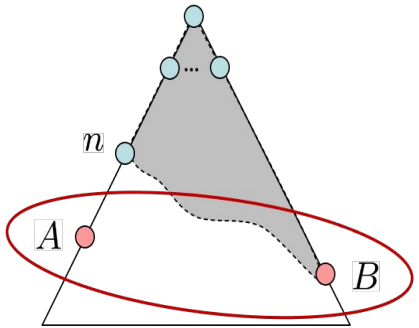
$f(n) = g(n) + h(n)$ Definition of f-cost

$f(n) \leq g(A)$ Admissibility of h

$g(A) = f(A)$ h = 0 at a goal

A* Optimality in Tree Search?

- Imagine B is on the fringe
- Some ancestor n of A is on the fringe,
- Maybe A itself is on the fringe too
- Claim: n will be expanded before B
  - f(n) is less or equal to f(A)
  - f(A) is less or equal to f(B)

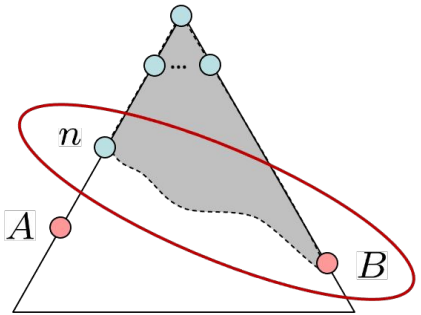


$g(A) < g(B)$ B is suboptimal

$f(A) < f(B)$ h = 0 at a goal

A* Optimality in Tree Search?

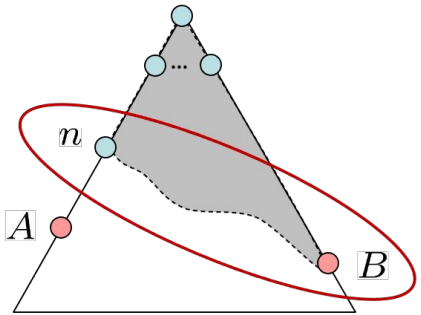
- Imagine B is on the fringe
- Some ancestor n of A is on the fringe,
- Maybe A itself is on the fringe too
- Claim: n will be expanded before B
  - f(n) is less or equal to f(A)
  - f(A) is less or equal to f(B)
  - n expands before B



$$f(n) \leq f(A) < f(B)$$

Hence …

- All ancestors of A expand before B
- A expands before B
- A* search is optimal



$$f(n) \leq f(A) < f(B)$$

Next Session: Consistency of Heuristics & Graph Search